

COMP2271: Computer Graphics

Part 1: Introduction, Data Science & The Rendering Pipeline

Dr. Frederick Li

Associate Professor

Department of Computer Science

Durham University

Section 1.1: What is Computer Graphics?

The Hidden World Behind the Glass

Before we define the field technically, let us begin with a thought experiment. Imagine for a moment that your screen is not a flat surface, but a window. You look through it and see a vast, 3D world with mountains, oceans, and characters. It feels real. But here is the truth: **None of it exists** .

Every millisecond, your computer is performing a magic trick. It takes a list of boring numbers (coordinates) and furiously paints millions of tiny coloured dots to *trick* your brain into seeing depth where there is only a flat plane . How we teach a rock (the silicon chip) to perform this artistic deception 60 times a second is the core story of this module.

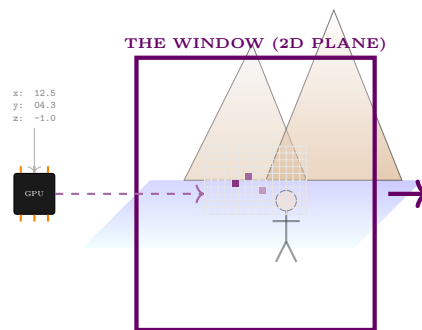


Figure 1: Visualising the rendering deception: Math $\rightarrow$ GPU $\rightarrow$ 2D Plane $\rightarrow$ Human Perception.

The Core Question

Computer Graphics is a vast and fascinating field of computer science dedicated to the generation, manipulation, and synthesis of visual content with the aid of computers. At its heart, the primary goal is to take an abstract description of a scene which might include shapes, materials, lights, and a camera and to systematically transform that description into a compelling 2D image.

This process is a powerful blend of art, physics, mathematics, and computational science. For example, we use the **physics** of light to simulate how it bounces around a scene to create realistic shadows and reflections. We rely heavily on the **mathematics** of linear algebra (vectors and matrices) to describe the position, orientation, and transformation of objects. We use algorithms rooted in **computational science** to perform these calculations efficiently. And finally, we use principles of **art** and cinematography to guide our choices of colour, composition, and camera angles to create an image that is not just correct, but also emotionally engaging.

Think of it as teaching a computer how to *see* and *draw* in a way that is meaningful and intuitive to humans. This technology is now ubiquitous, forming the backbone of countless applications. It is not just for the stunning special effects in cinematic blockbusters or immersive video games. It is fundamental to the fluid user interface on your smartphone, the life-saving visualisations of medical MRI data, the detailed virtual walkthroughs used by architects, and the complex

simulations used by scientists to understand everything from weather patterns to molecular structures.

How to Learn Computer Graphics: The Pipeline Effect

To master this introduction, focus on visualisation rather than rote memorisation. Additionally, beware the "Pipeline Effect". This module works sequentially. If you miss *Vertex Processing*, the subsequent *Fragment Processing* will not make sense .



Visualise the Flow: Treat the **Rendering Pipeline** as a factory assembly line. Track exactly how raw data inputs (vertices) flow through the stages to become a finished product (pixels).

The Three Pillars of Graphics

The field can be broadly understood through three fundamental and interconnected activities. While we list them sequentially, they often influence each other in a cyclical process.

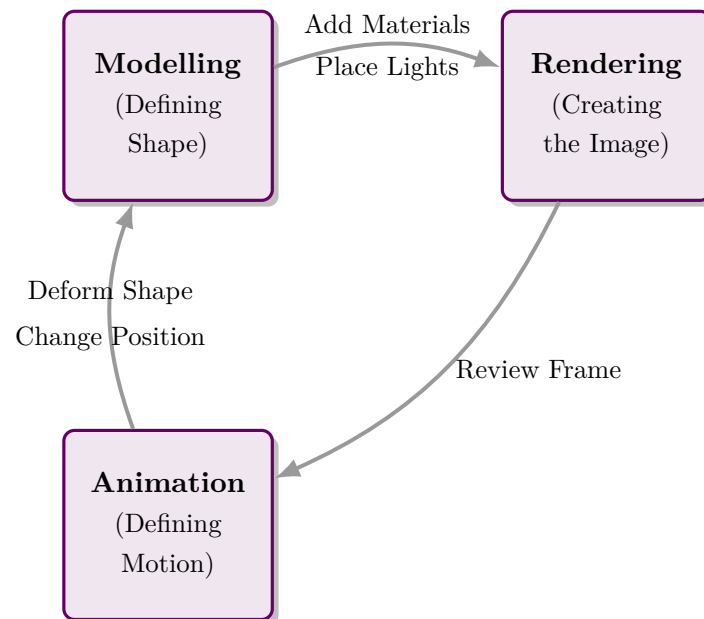


Figure 2: The interconnected relationship between the three pillars of computer graphics .

- **Modelling:** This is the process of creating a mathematical and geometric representation of a 3D object's surface. Modelling is concerned with defining the **shape and structure** of the objects we wish to visualise. This could be anything from a simple cube to a complex character model with intricate details. Techniques range from polygonal modelling (the most common in real-time graphics) to more complex methods like NURBS curves for smooth industrial designs. The output of this stage is typically a "mesh" of vertices, edges, and faces.
- **Rendering:** This is the process of generating a 2D image from a 3D model. It is the "drawing" or "photographing" phase, where the abstract model is brought to life. Rendering algorithms

must consider many factors to produce a realistic image, including the position and properties of light sources, the material characteristics of the objects (e.g., colour, shininess, roughness), and the position and settings of the virtual camera (e.g., its field of view).

- **Animation:** This is the process of creating the illusion of motion. In computer graphics, this is achieved by changing an object's properties such as its position, rotation, scale, or even its shape over time and rendering a sequence of images, or "frames". When these frames are displayed in rapid succession (e.g., 60 frames per second), the human eye perceives smooth movement. This can be done manually through keyframing or algorithmically through physics-based simulations.

Two Philosophies of Rendering

To achieve the goal of rendering, two main approaches have been developed. They differ fundamentally in how they approach the problem: one is object-centric, the other is image-centric.

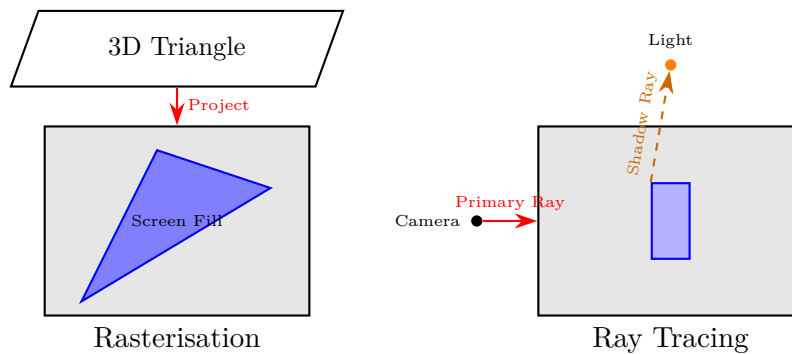


Figure 3: Visual comparison: Rasterisation projects geometry forward (Object $\rightarrow$ Screen); Ray Tracing casts rays backward (Camera $\rightarrow$ Scene) .

Rasterisation (Object-Order Rendering)

Rasterisation is the dominant approach for real-time applications like video games.

- **Analogy:** Imagine an artist painting on a canvas. They draw each object or shape one by one, and for each shape, they fill in the corresponding area on the canvas with colour. If one shape overlaps another, it is simply painted on top.
- **Process:** A rasterisation-based system takes 3D shapes (almost always triangles) and mathematically projects them onto the 2D plane of the screen. After determining where the triangle lands, the next step is to "fill it in," a process which involves figuring out exactly which pixels on the screen's grid are covered by that triangle.
- **Strength:** Its primary advantage is its incredible speed. GPUs are specifically designed to perform these calculations in parallel for millions of triangles per frame. This is the foundation of modern video games and the WebGL API we will be studying.

Ray Tracing (Image-Order Rendering)

Ray tracing is an alternative approach that is more closely aligned with the physics of light.

- **Analogy:** Instead of thinking about objects, think about how we see. Light rays travel from sources, bounce off objects, and eventually enter our eyes. Ray tracing simulates this process in reverse.
- **Process:** For each individual pixel on the screen, a ray tracing algorithm simulates a virtual ray of light traveling "backwards" from the camera, through that pixel, and out into the 3D scene. The algorithm then calculates what object this ray intersects with first and uses that information to determine the pixel's colour. By tracing additional rays (e.g., towards lights for shadows, or bouncing off surfaces for reflections), it can produce highly realistic images.
- **Strength:** Ray tracing excels at photorealism. Effects like perfect reflections, transparent refractions, and soft shadows are natural outcomes of the algorithm, whereas they are difficult to fake with rasterisation. Historically, it has been much slower, but modern GPUs now include dedicated hardware to accelerate it.

Course Focus

For the majority of this course, we will focus on the **Rasterisation** approach, as it is the core of all modern, real-time graphics hardware and APIs like WebGL.

Section 1.2: Computer Graphics & Data Science

Computer Graphics (CG) is not just for games; it is a critical component of the modern Data Science and AI pipeline, serving both as an **input generator** and an **output visualiser**.

CG as Input: Synthetic Data Generation

CG generates vast amounts of labelled training data for Deep Learning models when real-world data is scarce, expensive, or dangerous to collect.

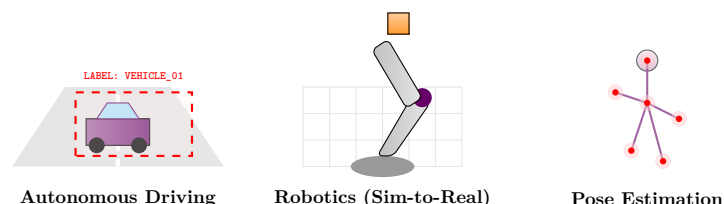


Figure 4: Examples of CG generating synthetic data for AI models.

- **Autonomous Driving:** Collecting real-world crash data is dangerous. By rendering photorealistic virtual cities, we know the exact position of every car and pedestrian (perfect "Ground Truth" labels) to train object detection networks (e.g., YOLO, R-CNN).

- **Robotics & Simulation:** Robots need millions of trials to learn to grasp objects. We use physics-based rendering to simulate robot arms interacting with 3D objects in a virtual environment (Sim-to-Real transfer).
- **Pose Estimation:** Rendering 3D human models in thousands of poses trains networks to recognise human body language from 2D video.

CG as Output: Advanced Visualisation

CG provides the tools to interpret complex model outputs, rendering high-dimensional data into human-understandable visual formats.

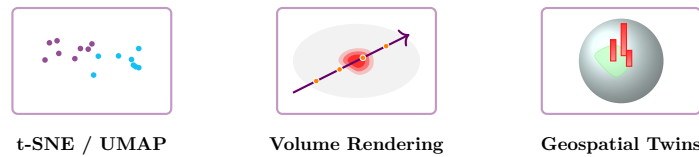


Figure 5: CG rendering techniques used to interpret complex AI outputs.

- **Dimensionality Reduction:** Transforming high-dimensional feature vectors into 3D scatter plots (e.g., t-SNE / UMAP). WebGL allows analysts to fly through millions of data points to find clusters and outliers in real-time.
- **Medical AI (Volume Rendering):** Deep learning models segment tumours in MRI scans. CG techniques (like Ray Marching) visualise these segmented 3D volumes, allowing doctors to see the exact shape and location of the pathology inside the body.
- **Geospatial Digital Twins:** Visualising AI predictions for climate change or traffic flow on a realistic 3D globe, using texture mapping and height maps to represent data density.

Section 1.3: The Rendering Pipeline

The 16-Millisecond Miracle

Before diving into the pipeline stages, we must address the constraint of **speed**. We have talked about algorithms, but to run a game at 60 frames per second, you have exactly **16.6 milliseconds** to draw one frame. In that blink of an eye, the GPU must take **2 million** triangles, transform every vertex, calculate lighting for **2 million** pixels, and figure out which pixels are hidden behind others. This is a logistical nightmare. Graphics programming is the art of organising this data so efficiently that the hardware doesn't choke.

A Conceptual Model

The **Rendering Pipeline** is a conceptual model that describes the sequence of steps a graphics system, particularly a GPU, follows to convert the raw data of a 3D scene into a final 2D image.

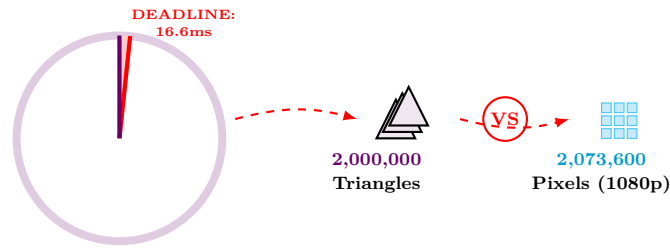


Figure 6: The logistical nightmare of processing millions of data points within 16.6 milliseconds.

It is not a single piece of hardware but rather a model of computation.

- **Analogy:** The most common analogy is a factory assembly line.
- **Input:** Raw materials (vertex data, textures, light positions, camera settings) are fed in at the start.
- **Process:** The data moves through a series of distinct stages. Some stages are fixed-function (their behaviour is built-in and unchangeable), while others are fully programmable using special programs called **shaders**. This programmability is the key to modern graphics, allowing developers to create custom visual effects.
- **Output:** At the end of the line, a finished product emerges—the final 2D image that is displayed on the screen (also known as the framebuffer).

The Role of the GPU

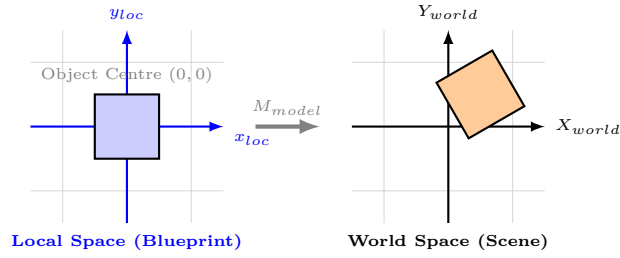
Modern Graphics Processing Units (GPUs) are highly specialised, massively parallel processors. They are engineered to execute the stages of this pipeline at incredible speeds, processing millions of vertices and pixels simultaneously. They are, in essence, the "factory" through which our graphics data is processed.

Deep Dive: The 6 Hardware Stages (Traditional Fixed-Function Pipeline)

Before discussing the modern programmable approach, it is crucial to understand the detailed hardware operations. While modern APIs abstract these, the underlying logic remains the same.

1. Object Transformation (Local $\rightarrow$ World Space)

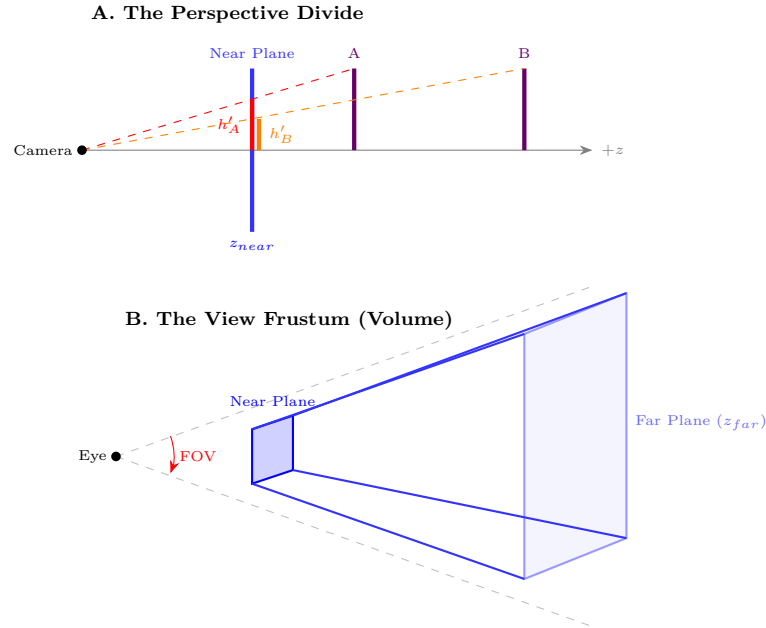
Vertices begin in **Local Space**, defined relative to the object's own centre (e.g., the nose of a character might be at $z = 1$). This stage multiplies every vertex by a **Model Matrix** to apply Translation, Rotation, and Scaling. This places the "blueprint" correctly into the shared **World Space** alongside all other objects.



- *Technical Concern:* Transforming millions of vertices individually on a CPU is slow.
- *Solution:* GPUs use **SIMD (Single Instruction, Multiple Data)** architectures to apply the same matrix operation to thousands of vertices simultaneously.

2. Perspective Projection (World $\rightarrow$ Clip Space)

This stage mimics the physics of a physical camera or the human eye. To render a 3D scene, we must define a **Virtual Camera** which establishes our vantage point. This camera defines a specific volume of space called the **View Frustum** truncated pyramid that determines exactly what is visible.

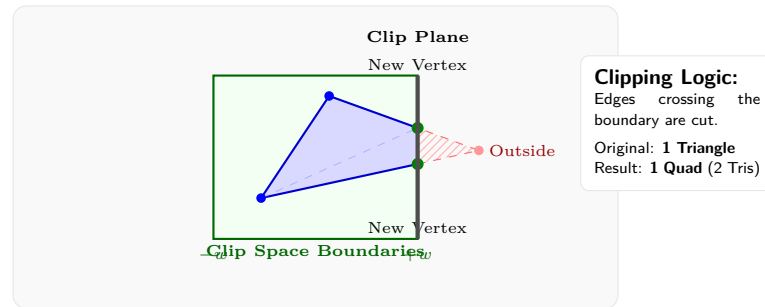


- **The View Frustum:** As shown in Diagram B, the camera cannot see infinitely. It is limited by a **Field of View (FOV)**, a **Near Plane** (the screen), and a **Far Plane** (draw distance). Objects must fall inside this truncated pyramid to be rendered.
- **The Perspective Divide:** Mathematically (Diagram A), the hardware divides the x, y, z coordinates by the depth component w . This causes objects further away (larger z) to be drawn closer to the optical axis (smaller x', y'), creating the illusion of depth.
- *Concern:* Compressing deep 3D space into a 2D plane can lead to precision errors known as **Z-Fighting**, where distant surfaces flicker because the GPU cannot distinguish their depths.

3. Clipping

Geometry outside the camera's field of view (the View Frustum) must be discarded to save

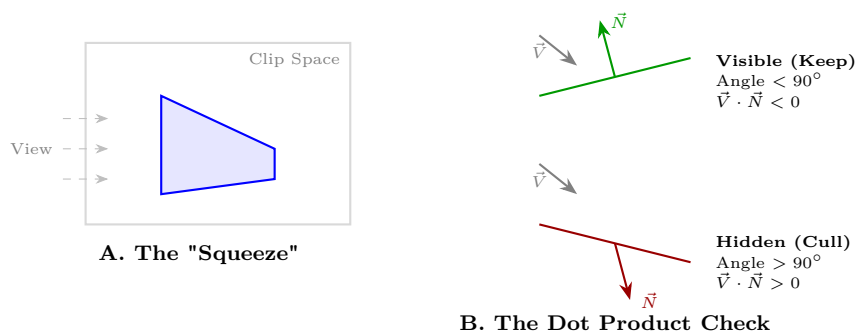
processing power. To maximise efficiency, this is done *after* projection, where the complex pyramid-shaped view frustum has been distorted into a simple **Unit Cube**.



- *Technical Concern:* What happens if a triangle is half-in and half-out? We cannot simply discard it, as that would create a hole in the mesh.
- *Solution:* Algorithms like **Sutherland-Hodgman** identify edges that cross the clip boundary. They calculate the exact intersection points, creating new vertices and re-connecting them to form a new polygon (which fits perfectly inside the screen) while discarding the portion outside.

4. Backface Culling

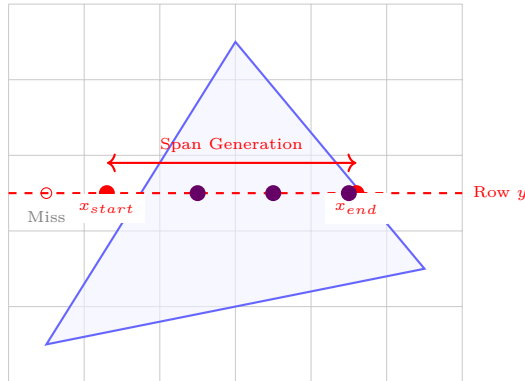
Most 3D objects are closed volumes (e.g., a solid sphere or cube). From any given angle, the viewer can only see the "front" half of the object; the "back" half is obscured by the front. Drawing these hidden faces is computationally wasteful. To optimize this, the hardware uses **Backface Culling** to discard polygons that face away from the camera before the expensive rasterisation stage.



- **The Mechanics (Diagram B):** Every polygon has a **Normal Vector** ($\vec{N}$) perpendicular to its surface. The hardware calculates the **Dot Product** between the view direction ($\vec{V}$) and the normal ($\vec{N}$).
- **The Check:**
 - If $\vec{V} \cdot \vec{N} < 0$, the face is pointing *towards* the camera (Visible).
 - If $\vec{V} \cdot \vec{N} > 0$, the face is pointing *away* from the camera (Hidden).
- **Efficiency:** In Clip Space (Diagram A), because the projection matrix makes all view rays parallel, this complex 3D check simplifies into a trivial 2D check known as **Winding Order**. The GPU simply checks if the vertices appear in Clockwise or Counter-Clockwise order on the screen. If the order is wrong, the triangle is discarded instantly, saving roughly 50% of the rendering load.

5. Rasterisation

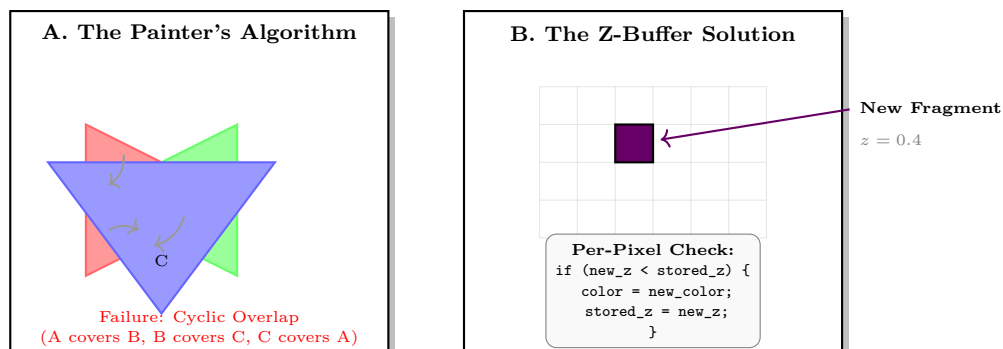
This is the bridge between continuous mathematics (shapes) and discrete electronics (pixels). It determines exactly which pixels on the monitor grid are covered by a triangle using a **Scanline Algorithm**.



- *Process:* The hardware sorts vertices by Y-coordinate, calculates the slope of the left and right edges, and iterates horizontally ("Span Generation") to emit **Fragments** for every pixel center that lies inside the edges.
- *Concern:* **Aliasing**. Because pixels are square, diagonal lines appear jagged (staircase effect).

6. Hidden Surface Removal & Shading

Finally, the system determines which pixels are visible and calculates their final colour. To understand why modern GPUs use a **Depth Buffer (Z-Buffer)**, it is helpful to look at the historical alternative.



- **Historical Approach (Painter's Algorithm):** Inspired by physical painting, early graphics systems tried to sort all polygons from farthest to nearest ($Z_{far} \rightarrow Z_{near}$) and draw them in that order.
 - *Why it failed:* Sorting millions of triangles per frame is computationally expensive ($O(N \log N)$). Worse, it fails completely on **Cyclic Overlaps** (as shown in Diagram A), where three triangles overlap each other in a circle. There is no mathematical "first" triangle to draw.
- **Modern Solution (Z-Buffer):** Modern hardware solves visibility at the **pixel level** rather than the object level.
 - The GPU maintains a **Depth Buffer** (a 2D array of floats matching the screen resolution) that stores the z -distance of the closest pixel drawn *so far*.

- *The Mechanism:* As rasterisation generates fragments (Diagram B), the hardware compares the fragment's depth (Z_{new}) against the value already in the buffer (Z_{stored}).
- **Rule:** If $Z_{new} < Z_{stored}$, the fragment is closer. It is drawn, and the buffer is updated. Otherwise, it is discarded.
- *Benefit:* This process is $O(1)$ per pixel and completely **order-independent**. You can submit triangles in any random order, and the final image will always be correct.

GPU Memory Architecture: Managing Time and Space

Beyond the processing pipeline, the GPU acts as a massive, high-speed memory manager. To ensure the resulting image is both visually stable (smooth motion) and geometrically correct (proper overlap), the GPU manages distinct "buffers", i.e., regions of VRAM dedicated to specific data types. Two of the most critical architectures govern the management of **Time** and **Space**.

1. Managing Time: Double Buffering (The Swap Chain)

A fundamental challenge in real-time graphics is that drawing takes time. If the GPU were to draw pixels directly to the screen while the monitor was in the middle of refreshing (scanning from top to bottom), the user would see a "half-finished" frame. The top half of the screen might show Frame N , while the bottom shows Frame $N + 1$. This visual artifact is known as **Screen Tearing**.

The Solution: We decouple the *drawing* process from the *display* process using **Double Buffering**.

- **Back Buffer (Private):** The GPU draws to this invisible memory buffer. It can take as long as it needs (e.g., 16ms) to render the scene. The user never sees this work-in-progress.
- **Front Buffer (Public):** This buffer contains the completed previous frame. The monitor hardware reads exclusively from this buffer to send colour signals to the pixels on the display.
- **The Swap:** When the GPU finishes drawing the Back Buffer, and the monitor indicates it is ready for a new frame (a signal known as **VSync** or Vertical Sync), the hardware essentially "flips a switch". The pointers to the buffers are swapped instantly. The old Back Buffer becomes the new Front Buffer, and the GPU begins clearing and drawing to the old Front Buffer.

2. Managing Space: The Z-Buffer (Depth Testing)

As discussed in the "Deep Dive," ensuring objects overlap correctly is a critical problem. The **Z-Buffer** (or Depth Buffer) is the hardware solution that manages 3D space in a 2D memory array.

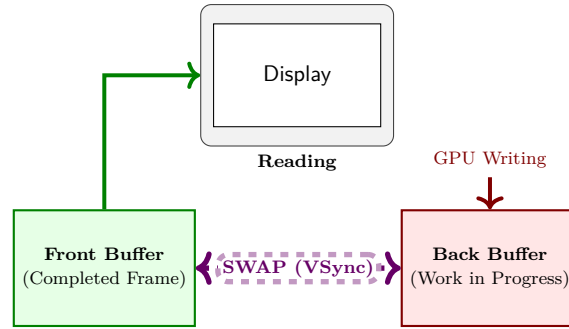


Figure 7: The Double Buffering mechanism. The "Swap" ensures the monitor only ever sees a complete image.

How to Operate:

1. **Allocation:** Alongside the standard Colour Buffer (RGB), the GPU allocates a Depth Buffer (Float) of the exact same resolution (e.g., 1920×1080).
2. **Initialisation:** At the start of every frame, the Depth Buffer is cleared to the maximum distance ($z = 1.0$ or ∞), representing "infinite depth."
3. **The Test:** As the Rasteriser generates a fragment at pixel (x, y) , it calculates that fragment's depth value (Z_{new}). It reads the value currently stored in the buffer at (x, y) (Z_{stored}).
4. **Decision:**
 - **IF** $Z_{new} < Z_{stored}$: The new fragment is **closer** to the camera. The GPU writes the new colour to the Colour Buffer and updates the Depth Buffer with the new Z value.
 - **ELSE**: The new fragment is **behind** something we have already drawn. The fragment is discarded immediately.

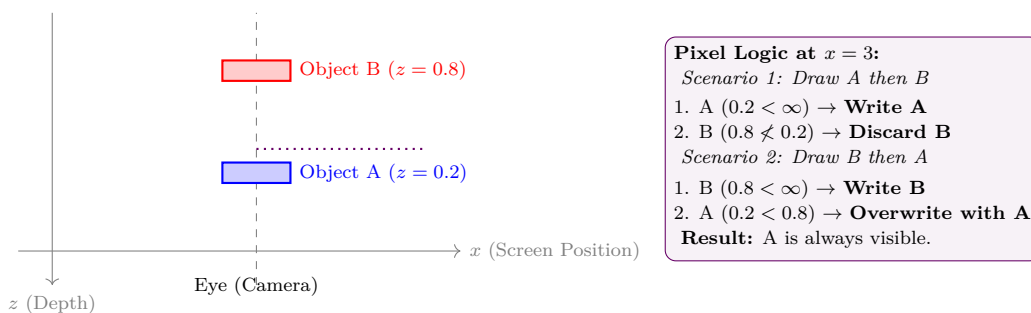


Figure 8: Visualising the Depth Test. Regardless of draw order, the object with the lower Z value (closer to 0) wins.

3. The Cost of Rasterisation: Memory Analysis

Understanding these architectures allows us to calculate the raw memory cost of a graphics application. Ideally, VRAM is needed not just for textures and geometry, but for the fundamental framebuffer themselves.

To render a standard 1080p frame (1920×1080 pixels), the GPU needs:

- **Front Buffer (Colour):** 4 Bytes per pixel (Red, Green, Blue, Alpha channels, 8 bits each).
- **Back Buffer (Colour):** 4 Bytes per pixel.
- **Depth Buffer (Z-Depth):** 4 Bytes per pixel (32-bit Float precision).

Total VRAM Formula:

$$\begin{aligned}\text{Memory} &\approx \text{Pixels} \times (\text{Colour}_{\text{Front}} + \text{Colour}_{\text{Back}} + \text{Depth}) \\ &\approx 2,073,600 \text{ px} \times (4\text{B} + 4\text{B} + 4\text{B}) = 2,073,600 \times 12 \text{ Bytes} \approx \mathbf{24.9 \text{ MB}}\end{aligned}$$

This 25MB is the absolute *minimum* "cost of entry" just to open a 1080p window, before loading a single texture or model. For 4K resolution (3840×2160), this base cost jumps to nearly **100 MB**, illustrating why resolution has such a high impact on GPU performance and memory pressure.

Bridging the Views: From Hardware to Shaders

Modern APIs like WebGL abstract these hardware steps into a programmable workflow. The "traditional" steps listed above do not disappear; instead, they are grouped and exposed to the developer via **Shaders**.

- **Vertex Processing (Vertex Shader):** The programmer writes a Vertex Shader that takes over the responsibility of **Object Transformation** and **Perspective Projection** (Steps 1 & 2). You explicitly write the matrix math to position your vertices.
- **Fixed Function Hardware:** The GPU automatically handles **Clipping**, **Backface Culling**, and **Rasterisation** (Steps 3, 4, & 5). These remain highly optimised hardware functions that the programmer generally does not write code for.
- **Fragment Processing (Fragment Shader):** The programmer writes a Fragment Shader to handle the colour calculations (**Shading**). The **Hidden Surface Removal** (Step 6) is typically handled automatically by the hardware via the Depth Buffer after the fragment shader runs.

The Modern Programmable Pipeline (Visualised)

The modern WebGL pipeline can be broken down into three high-level conceptual stages. Two of these are programmable using special C-like programs called **shaders**.

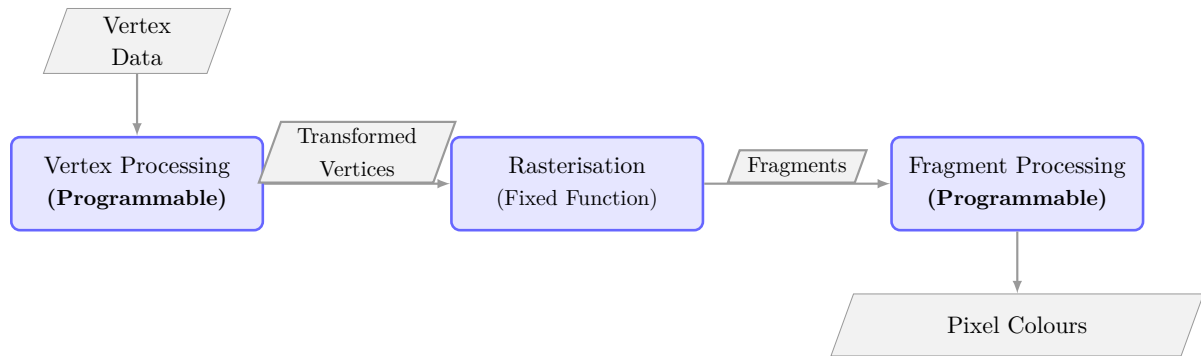


Figure 9: Data flows from vertices to fragments (potential pixels) to final pixel colours .

Stage 1: Vertex Processing (via the Vertex Shader)

This is the first programmable stage. Its primary goal is to process the individual points (**vertices**) that constitute our 3D models.

- The GPU executes a **Vertex Shader** program that you write. This program runs once for every single vertex in your scene.
- **Key Responsibilities:**
 - **Transformation:** Its most critical job is to transform the 3D position of a vertex from its local "model space" (coordinates relative to the model's own origin) through various coordinate systems (world space, camera space) and ultimately into the final 2D screen position ("clip space"). This involves a series of matrix multiplications that account for the model's position, the camera's viewpoint, and the perspective effect.
 - **Data Passthrough:** It also processes and passes along other per-vertex data (like texture coordinates or normals, which describe the direction the surface is facing) to the next stage in the pipeline.

Stage 2: Rasterisation (Fixed Function)

This stage is a fixed, non-programmable part of the hardware. Its job is to take the transformed vertices from the previous stage and determine which pixels on the screen are covered by the geometric shapes (primitives) they form.

- The GPU connects the vertices to form primitives, which are almost always triangles.
- The rasteriser then "fills in" these triangles, iterating over the 2D grid of pixels and generating a "fragment" for each pixel that lies inside the triangle's boundary. A fragment is a candidate pixel; it holds all the information needed to calculate the final colour of that pixel.
- During this stage, per-vertex attributes (like colour or texture coordinates) are smoothly **interpolated** across the surface of the triangle for each fragment. For example, if one

vertex is red and another is blue, the fragments along the edge between them will be a smooth gradient of purple shades.

Stage 3: Fragment Processing (via the Fragment Shader)

This is the second programmable stage, and it is where the visual richness of a scene is created.

- The GPU executes a **Fragment Shader** program that you write. This program runs once for every single fragment generated by the rasteriser.
- **Key Responsibility:** Its sole purpose is to calculate and output the final colour for a single fragment.
- **Common Operations:** This is where the magic happens. The fragment shader uses the interpolated data from the rasteriser to perform tasks like:
 - **Texturing:** Looking up a colour from a 2D texture image.
 - **Lighting:** Performing calculations to simulate how light interacts with the surface, creating highlights, shadows, and definition.
 - **Special Effects:** Implementing any number of effects like fog, transparency, or stylised outlines to determine the final pixel colour.

Section 1.4: Practical Implementation with WebGL

WebGL (Web Graphics Library) is a JavaScript API that exposes a low-level graphics pipeline, based on the OpenGL ES standard, directly within a web browser. It allows us to write GLSL shaders and control the GPU to render complex 3D scenes on an HTML `<canvas>` element. It gives us fine-grained control, but requires us to manage every step of the pipeline manually.

The HTML Boilerplate

All WebGL graphics are drawn onto a standard HTML `<canvas>` element. This provides the drawing surface for our application.

```
<!DOCTYPE html>
<html>
  <head>
    <title>WebGL Boilerplate</title>
  </head>
  <body>
    <canvas id="glCanvas" width="640" height="480"></canvas>
    <script src="main.js"></script>
  </body>
</html>
```

Analysis

The `id` attribute is crucial as it provides a handle to access the canvas element from our JavaScript code. The `<script>` tag, placed at the end of the body, loads our main program where all the WebGL logic will reside.

Getting the WebGL Rendering Context

The first step in any WebGL program is to get the "rendering context" from the canvas. This context object is our sole interface for communicating with the GPU; all graphics commands will be called through it.

```
// Get the canvas element from the HTML document
const canvas = document.getElementById('glCanvas');
// Get the WebGL rendering context
const gl = canvas.getContext('webgl');

if (!gl) {
    alert('Unable to initialize WebGL. Your browser may not support it.');
```

Analysis

The line `canvas.getContext('webgl')` is the most important call here. It requests a WebGL rendering context from the canvas. If the browser does not support WebGL, it will return `null`. The resulting `gl` object holds all the WebGL functions and constants we will use (e.g., `gl.clearColor`, `gl.createBuffer`).

A Basic Vertex Shader

Shaders are written in a C-like language called **GLSL** (OpenGL Shading Language). The vertex shader's minimum responsibility is to set the special built-in variable `gl_Position`.

```
// An 'attribute' is an input to the vertex shader.
// It receives data from a GPU buffer we set up in JavaScript.
attribute vec2 a_position;

void main() {
    // gl_Position is a special, required output variable.
    // It determines the final position of the vertex in "Clip Space".
    // WebGL requires a 4D vector (x, y, z, w) for this.
    gl_Position = vec4(a_position, 0.0, 1.0);
}
```


Analysis

`attribute vec2 a_position;` defines an input named `a_position`. A `vec2` is a 2-component vector (x, y). The data for this attribute will be supplied from a buffer we create in JavaScript. `gl_Position` is the critical output; we must assign a 4D vector to it. We construct this by taking our 2D input position, setting the z-coordinate to 0.0, and the w-coordinate to 1.0 (which is standard for positional vectors).

A Basic Fragment Shader

The fragment shader's minimum responsibility is to set the special built-in variable `gl_FragColor`, which defines the final colour of a pixel.

```
// This sets the default precision for floating point numbers.
// It's a mandatory line in WebGL fragment shaders.
precision mediump float;

void main() {
    // gl_FragColor is a special, required output variable.
    // It determines the final color and expects a 4D vector
    // representing (Red, Green, Blue, Alpha).
    gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0); // Solid, opaque Red
}
```

Analysis

`precision mediump float;` is a required boilerplate line that sets the default precision for floating-point calculations. `gl_FragColor` is the critical output for this shader. We assign it a `vec4` where each component represents a colour channel (R, G, B, A) with values ranging from 0.0 (off) to 1.0 (full intensity).

Section 1.5: A Higher-Level Approach with Three.js

While WebGL gives us direct access to the rendering pipeline, it is very verbose. For every object we want to draw, we need to manually set up buffers, write shaders, manage shader programs, and calculate matrix transformations. This is powerful but often tedious.

Three.js is a JavaScript library that provides a much higher level of abstraction. It sits on top of WebGL and handles all the low-level, boilerplate code for us. Instead of thinking about vertices, fragments, and buffers, we get to work with intuitive concepts like 'Scene', 'Camera', 'Mesh', 'Material', and 'Light'.

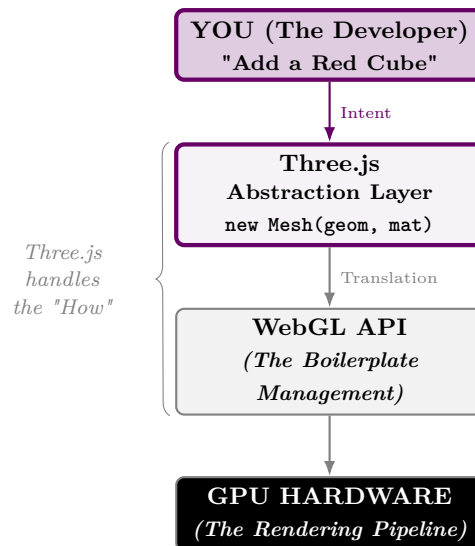


Figure 10: Three.js elevating creative control over technical WebGL complexity.

Three.js Example: A Spinning Cube

Let's see how to create the same result as a complex WebGL program drawing a shape on screen with just a few lines of Three.js code.

```

<!DOCTYPE html>
<html>
  <head>
    <title>Three.js Spinning Cube</title>
    <style>
      body { margin: 0; }
      canvas { display: block; }
    </style>
  </head>
  <body>
    <script type="importmap">
      {
        "imports": {
          "three": "https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.module.js"
        }
      }
    </script>
    <script type="module">
      import * as THREE from 'three';

      // 1. Scene Setup
      const scene = new THREE.Scene();
      const camera = new THREE.PerspectiveCamera(75,
        window.innerWidth / window.innerHeight, 0.1, 1000);
      const renderer = new THREE.WebGLRenderer();
      renderer.setSize(window.innerWidth, window.innerHeight);
      document.body.appendChild(renderer.domElement);
    </script>
  </body>
</html>
  
```

```

// 2. Object Creation
const geometry = new THREE.BoxGeometry(1, 1, 1);
const material = new THREE.MeshBasicMaterial({ color: 0x00ff00 }); // Green
const cube = new THREE.Mesh(geometry, material);
scene.add(cube);

camera.position.z = 5;

// 3. Render Loop
function animate() {
  requestAnimationFrame(animate);

  // Animation logic
  cube.rotation.x += 0.01;
  cube.rotation.y += 0.01;

  renderer.render(scene, camera);
}

animate();
</script>
</body>
</html>

```

Analysis of the Three.js Code

1. **Scene Setup:** This block creates the three essential components.

- `new THREE.Scene()`: This object acts as a container for everything we want to render all our models, lights, and cameras.
- `new THREE.PerspectiveCamera(...)`: This defines our viewpoint. It mimics a real-world camera, creating the effect where objects farther away appear smaller. The arguments control field of view, aspect ratio, and clipping planes.
- `new THREE.WebGLRenderer()`: This is the workhorse. It's the object that will take the 'scene' and 'camera' we provide and perform all the underlying WebGL calls to render the image onto an HTML '<canvas>' element (which it creates for us).

2. **Object Creation:** Here we define the object we want to see.

- `new THREE.BoxGeometry(...)`: This defines the shape of our object. It's a set of vertices and faces that form a cube. This is the "Modelling" pillar. This data will eventually be sent to the GPU's vertex buffer.
- `new THREE.MeshBasicMaterial(...)`: This defines the appearance of the object's surface. A 'MeshBasicMaterial' is the simplest type; it's not affected by lights. It essentially tells the fragment shader, "just draw this solid colour". The colour '0x00ff00' is hexadecimal for green.

- `new THREE.Mesh(geometry, material)`: A 'Mesh' is the final object that combines a shape (geometry) with an appearance (material). We then add this 'Mesh' to our 'scene'.
3. **Render Loop:** The 'animate' function is our render loop, powered by 'requestAnimationFrame' just like in our low-level WebGL example.
- Inside the loop, we update the cube's 'rotation' properties. This modifies the model's transformation matrix for the next frame, fulfilling the "Animation" pillar.
 - The most important line is `renderer.render(scene, camera)`. This single command tells Three.js to execute the "Rendering" pillar. It takes the entire scene graph, sets up the appropriate camera and projection matrices, compiles the necessary GLSL shaders based on the materials used, sends all the vertex data to the GPU, and issues the draw calls. All the complexity of the rendering pipeline is handled for us by this one function call.

Section 1.6: The Hardware Behind the Pixels

To truly master computer graphics, one must look beyond the code and understand the machinery that drives it. The rendering process is a high-speed relay race between two distinctly different processors: the **CPU** (Central Processing Unit) and the **GPU** (Graphics Processing Unit).

While they look similar on a spec sheet, their internal architectures are designed for completely opposing goals: *Latency* versus *Throughput*.

Architect vs. Workforce: The Design Philosophy

We can visualise the difference using the metaphor of an **Architect** and a **Construction Crew**.

- **The CPU (The Architect - Latency Oriented):** The CPU is designed to minimise **Latency** (the time it takes to complete a single task). It features a small number of incredibly powerful cores clocked at high speeds.
 - It has large local caches (L1/L2/L3) to keep data close.
 - It uses complex control logic for "Branch Prediction" to guess the outcome of `if/else` statements before they happen.
 - *Role:* It handles the "Scene Graph" the complex, serial logic of your application (physics, AI, input handling).
- **The GPU (The Workforce - Throughput Oriented):** The GPU is designed to maximise **Throughput** (the total amount of work done per unit of time). It sacrifices individual core speed and complex control logic to fit thousands of smaller, simpler cores onto the silicon.

- It uses a **SIMD** (Single Instruction, Multiple Data) architecture. This means one instruction (e.g., "multiply by 2") is issued once but executed simultaneously by 32 or 64 cores on different data points.
- It relies on massive memory bandwidth (GDDR VRAM) to feed these hungry cores.
- *Role*: It processes the "Geometry and Pixels" massive parallel tasks where every pixel is treated identically .

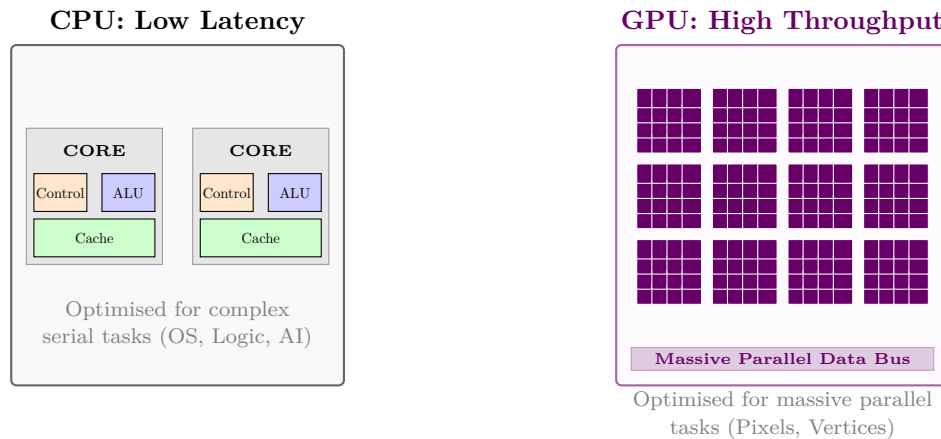


Figure 11: Hardware Architecture Comparison. The CPU (left) relies on large caches and complex control logic. The GPU (right) relies on sheer numbers of compute units (ALUs) working in parallel.

The Integration: Anatomy of a Render Call

The collaboration between these two processors defines the rendering pipeline. In a typical WebGL or Three.js application, this interaction is a constant cycle of data preparation and hand-off:

1. **CPU Stage (The Logic Loop):** Your JavaScript code runs here. Every frame, the CPU updates the **Scene Graph**, calculating the new position of a car, the rotation of a camera, or the skeleton of a character. It packages this mathematical state into matrices .
2. **The Bridge (The Bottleneck):** When you call `renderer.render()`, the CPU must transfer this data across the PCIe bus (or internal memory bus) to the GPU's VRAM. This transfer is relatively slow compared to processing speed, which is why we try to minimise the number of "Draw Calls" per frame .
3. **GPU Stage (The Execution):** Once the data is in VRAM, the GPU takes over. It launches thousands of threads. The **Vertex Shader** transforms points, and the **Fragment Shader** colours pixels. Because of the GPU's throughput architecture, it can hide the latency of memory access: while one group of pixels is waiting for a texture to load, the GPU simply switches to processing the next group of pixels instantly .

Section 1.7: Coding Paradigms: WebGL vs. Three.js

We have established that Three.js is built *on top* of WebGL. However, the difference is not just about writing "less code" it is a fundamental shift in **coding paradigm**.

We move from an **Imperative** model (telling the computer *how* to do something step-by-step) to a **Declarative** model (telling the computer *what* you want) .

Raw WebGL: The State Machine

OpenGL (and by extension WebGL) acts as a global **State Machine**. You set a specific "state" for example, "use this buffer" or "use this shader" and that state persists globally until you explicitly change it. This makes it powerful but dangerous; if you forget to "unbind" a buffer, it might accidentally be used by the next object you try to draw.

In raw WebGL, you are the factory manager. You must:

- Manually allocate and bind memory buffers.
- Compile shader source code strings and check for compilation errors.
- Link shader programs.
- Define byte-level memory layouts (e.g., "tell the GPU that the first 3 numbers are position, and the next 2 are UVs") .

Three.js: The Object-Oriented Abstraction

Three.js wraps this state machine in an **Object-Oriented** model. You create a **Mesh** object, and that object "knows" its own geometry and material. When you ask Three.js to render that mesh, the library automatically handles the complex state switching behind the scenes .

The "Triangle Test": A Code Comparison

The standard "Hello World" of graphics is drawing a single triangle. Comparing the code required for this simple task reveals the massive abstraction layer Three.js provides.

```
// RAW WebGL: ~50+ Lines (Imperative)
// 1. You must manually compile the shader source code
const vShader = gl.createShader(gl.VERTEX_SHADER);
gl.shaderSource(vShader, `attribute vec2 p; void main(){ gl_Position=vec4(p,0,1); }`);
gl.compileShader(vShader);

// 2. You must manually link them into a "Program"
const program = gl.createProgram();
gl.attachShader(program, vShader);
gl.attachShader(program, fShader);
```

```

gl.linkProgram(program);

// 3. You must manually manage GPU memory (Buffers)
const buffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
// We have to use TypedArrays to talk to the GPU
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array([0,1, -1,-1, 1,-1]), gl.STATIC_DRAW);

// 4. You must tell the GPU exactly how to read that memory
const loc = gl.getAttribLocation(program, "p");
gl.enableVertexAttribArray(loc);
gl.vertexAttribPointer(loc, 2, gl.FLOAT, false, 0, 0);

// 5. Finally, draw
gl.drawArrays(gl.TRIANGLES, 0, 3);

```

```

// THREE.JS: ~10 Lines (Declarative)

// 1. Define the Geometry (The Shape)
const geometry = new THREE.BufferGeometry();
const vertices = new Float32Array([0,1,0, -1,-1,0, 1,-1,0]);
geometry.setAttribute('position', new THREE.BufferAttribute(vertices, 3));

// 2. Define the Material (The Appearance)
// Three.js generates and compiles the shader code automatically!
const material = new THREE.MeshBasicMaterial({ color: 0xff0000 });

// 3. Combine into a Mesh and Add to Scene
const mesh = new THREE.Mesh(geometry, material);
scene.add(mesh);

// 4. Render
renderer.render(scene, camera);

```

The Abstraction Cost

While Three.js boosts productivity, it introduces an **Abstraction Cost**.

- **Hidden Overhead:** Every `Mesh` you create in Three.js adds CPU overhead to manage its state. Drawing 10,000 individual `THREE.Mesh` objects will be significantly slower than drawing 10,000 triangles in a single raw WebGL buffer.
- **The "Black Box":** If performance drops, you might not know why, because the complexity of the draw calls is hidden from you. This is why we study the underlying pipeline.

Section 1.8: Under the Hood: The Architecture of Three.js

While we have seen how to write Three.js code, understanding how it executes is crucial for performance optimisation. Three.js is not merely a wrapper around WebGL functions; it is a

complex engine that acts as both a **Scene Graph Manager** (on the CPU) and a **Shader Factory** (for the GPU).

The CPU-GPU Relay Race

When you create a Three.js application, you are effectively setting up a relay race between the CPU and the GPU.

1. **The Director (CPU):** Your JavaScript code manages the high-level logic. You create a hierarchy of objects (the Scene Graph), update their positions using physics or input logic, and define their appearance using Materials. This all happens in the browser's main memory (RAM).
2. **The Translation (Three.js Core):** When you call `renderer.render()`, Three.js traverses your scene graph. It calculates the final world positions of every object (multiplying local matrices by parent matrices) and determines which objects are visible to the camera.
3. **The Transfer (The Bus):** The engine then packs this data (vertex positions, normals, texture images, and transformation matrices) and uploads it to the GPU. This "draw call" overhead is often the performance bottleneck in web graphics.
4. **The Artist (GPU):** Finally, the GPU receives the data and executes the shaders to draw the pixels on the screen.

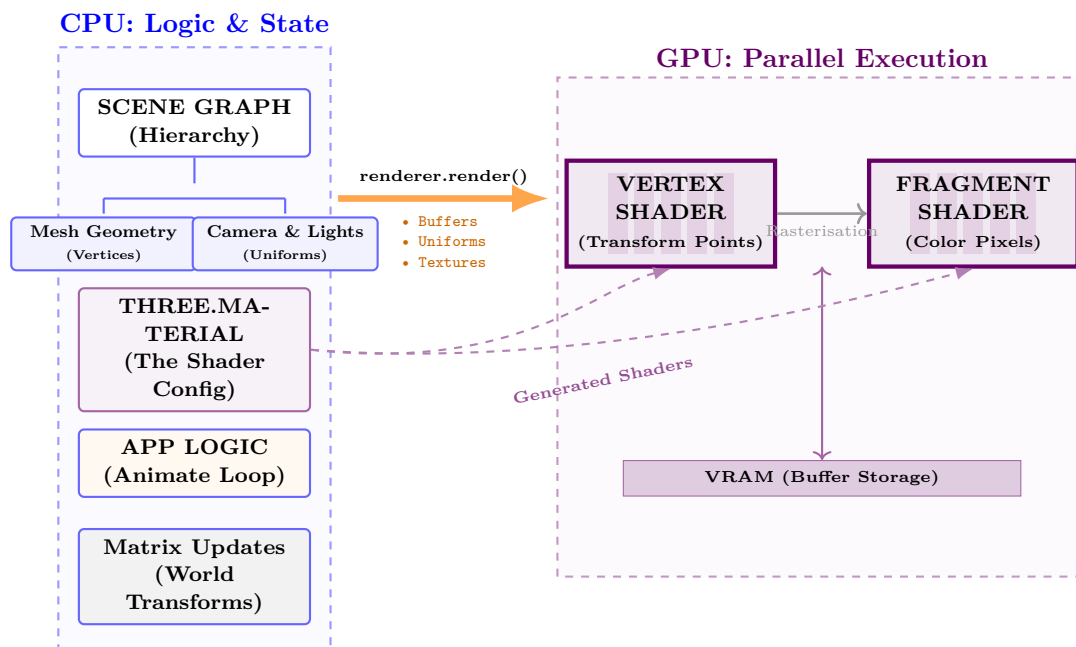


Figure 12: The Anatomy of a Three.js Program. The CPU manages the state (left), while the GPU executes the massive parallel processing (right). Three.js handles the complex data transfer between them.

The Material System: A Shader Factory

One of the most powerful "hidden" features of Three.js is its Material system. In raw WebGL, you must write GLSL shader code (strings of C-like code) by hand. In Three.js, you simply define properties in a JavaScript object:

```
const material = new THREE.MeshPhongMaterial({
  color: 0xff0000,
  map: texture,
  shininess: 30
});
```

Behind the scenes, Three.js acts as a **Shader Factory**. It maintains a library of shader "chunks" (snippets of GLSL code). When you create a material, the engine dynamically "stitches" these chunks together based on the properties you enabled.

- If it sees the `map` property, it injects the `#define USE_MAP` pre-processor directive and the corresponding texture lookup code.
- If it sees `shininess`, it injects the specular lighting math calculations.

This allows you to benefit from the performance of highly optimized, custom GPU shaders without having to write a single line of GLSL yourself. However, understanding this generation process is vital for debugging. If you set a property incorrectly (e.g., passing the wrong data type), you aren't just setting a variable wrong; you are effectively breaking the GLSL code generation process, which can result in silent rendering failures where your object simply disappears or renders as black .

Section 1.9: Reflective Questions: Why Learn Computer Graphics?

As we conclude this introduction, consider the following questions. They are designed not to test your memory, but to stimulate your curiosity and motivation for the modules ahead.

- **The Art of Deception:** We learned that Rasterisation is an "object-order" process, effectively painting shapes onto a screen. This is fundamentally different from how light works in the real world (rays hitting the eye). *Why, then, do modern video games look so realistic? What "tricks" must we be using in our shaders to fake lighting and materials so convincingly that the human eye accepts them?*
- **The 16-Millisecond Challenge:** To achieve a smooth 60 frames per second, the entire Rendering Pipeline from the CPU processing graphics application logic to the GPU colouring the final pixel must finish in roughly 16 milliseconds. *How is it possible to process millions of triangles and pixels in such a short window? What does this tell you about the importance of efficient algorithms and the architecture of the GPU?*
- **The Power of the Pixel:** In the programmable pipeline, you write the code that determines the colour of every single pixel. *If you had total control over light and colour, what visual style would you create? Would you aim for hyper-realism, or would you write a shader that makes 3D objects look like a hand-drawn cartoon?*
- **Abstraction vs. Understanding:** We saw that Three.js makes creating a cube easy (5 lines of code), while raw WebGL is hard (50+ lines). *Why is it valuable for a Computer Scientist to understand the "hard way"? How does knowing what happens in the Vertex and Fragment stages help you debug a Three.js scene that isn't rendering correctly?*

Looking Ahead

In the coming weeks, we will peel back the layers of abstraction. We will move from simple shapes to complex hierarchies, and from flat colours to programmable lighting models. The goal is to transform you from a consumer of graphics APIs into a creator of visual computing solutions.